

G05 · Atlas

G05 · Atlas — 문맥을 더 잘기억하기

이 권은 Titans·Miras 계열이 남긴 세 가지 아쉬움을 Atlas 가 어떻게메우는지 **쉽게** 다시 쓴다. 핵심은 딱 세 가지다 — 토큰 하나가 아니라 **최근 여러 토큰을 한꺼번에** 기억하고 (Omega rule), 메모리의 **용량을 키우고** (feature map), 갱신을 **더 똑똑하게** 한다 (Muon). 우리 목표는 증명이 아니라, 이 세 손잡이가 state 크기와 토큰당 연산에 어떤 청구서를 남기는지 — 즉 시스템 모델링에 바로 쓸 수 있는 그림 — 을 얻는 것이다.

직관 이 계열의 메모리는 “읽으면서 계속 학습시키는 작은 모델”이다. 토큰이들어올 때마다 그 작은 모델을 조금 갱신해 문맥을 눌러 담는다. Titans 까지는 이 작은 모델을 **한 번에 토큰 하나**로만 가르쳤다. Atlas 는 같은 아이디어를 유지하되, 가르치는 방식 (무엇을 · 얼마나 · 어떻게) 을 세 군데서 업그레이드한다.

1. 왜 Atlas 인가 — 앞 권이 남긴 세 가지 아쉬움

Atlas 논문 (Atlas: Learning to Optimally Memorize the Context at Test Time, arXiv:2505.23735) 이 스스로 지목한, 지금까지의 recurrent memory 모델이 가진 세 가지 결함은 이렇다.

1. **한 토큰씩 갱신 (online 갱신)**. 메모리가 지금 들어온 토큰 하나만보고 갱신된다. “직전 여러 토큰이 **함께** 잘 저장됐는가” 는 아무도 묻지 않는다. 개별 토큰은 하나하나 안 놀라워도, 여러 토큰이 뭉친 사건은 통째로 기억할 가치가 있을 수 있는데, 토큰 하나짜리 목표는 이 경우를 놓친다.
2. **제한된 용량 (capacity)**. 고정 크기 state 에 문맥을 압축한다지만, 그 state 가 **몇 개의 연상 (association) 을 저장할 수 있는**지가 구조적으로 막혀 있다. 파라미터를 늘려도 용량이 그만큼 늘지 않는다.
3. **1차 갱신만 사용**. inner optimizer 가 거의 전부 plain gradient descent (1차) 다. gradient 의 방향 정보만 쓰니 나쁜 지점에 눌러앉아 질 낮은 key → value 매핑을 저장할 수 있다.

비유 메모장에 회의 내용을 받아적는 비서를 생각하자. (1) 이 비서는 한 문장들으면 바로 한 줄 적고 다음 문장으로 넘어간다 — 문단 전체의 맥락을 못본다. (2) 메모장이 작아서 일정 개수 이상은 적으면 앞엿것이 뭉개진다. (3) 받아적는 규칙이 단순해서 (들리는 대로만) 중요한 대목을 강조하거나 오타자를 교정하지 못한다. Atlas 는 이 셋을 각각 고친다.

Atlas 는 이 세 결함에 세 해법을 정확히 하나씩 대응시킨다.

결함	해법	한 줄 요약
한 토큰씩 갱신 용량 한계	Omega rule feature map (ϕ)	최근 c 개 토큰을 한꺼번에 기억 key 를 더 큰 공간으로 올려 저장 칸을 늘림

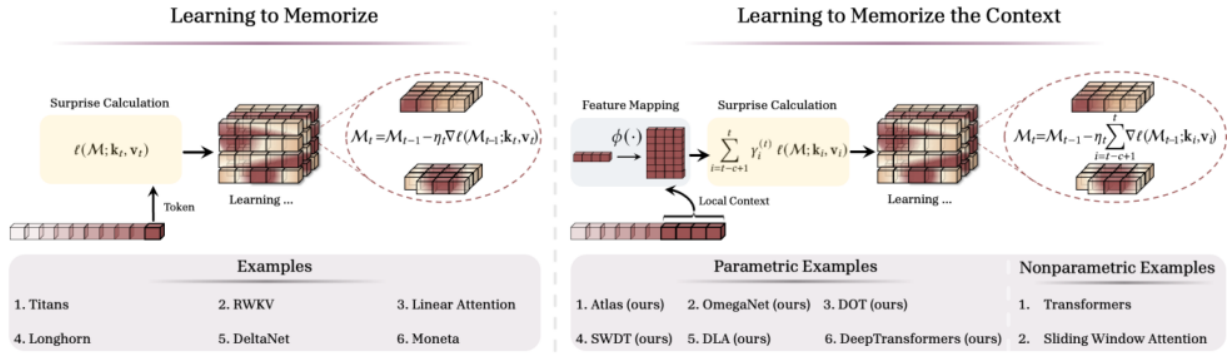


그림 1 — 개별 토큰을 기억하기 (왼쪽) 와 문맥을 기억하기 (오른쪽) 의 대비. 왼쪽은 지금 토큰 하나의 surprise 로 메모리를 갱신하고, 오른쪽은 feature map 으로 올린 뒤 최근 c 개 토큰을 한꺼번에 최소화한다 (Omega rule). 오른쪽 맨 끝의 “Nonparametric Examples” 가 Transformer·SWA 다. 출처: 저자, Atlas(arXiv:2505.23735) Fig.1 — 원저자 그림 (본서 결과 아님).

결합	해법	한 줄 요약
1 차 갱신	Muon	근사 2 차 정보로 더 똑똑하게 갱신

핵심 Atlas = (Omega rule 로 무엇을 기억할지) + (feature map 으로 얼마나 많이 기억할지) + (Muon 으로 어떻게 갱신할지) 를 동시에 손본 모델. 세 손잡이가 서로 독립이라는 게 설계의 골격이다.

2. 해법 1 — Omega rule: 토큰 하나 말고 최근 여러 토큰을 한꺼번에

먼저 그림 하나로 전체 대비를 잡자. 왼쪽은 “토큰 하나를 기억하는” 옛방식, 오른쪽은 “문맥 (최근 여러 토큰) 을 기억하는” Atlas 방식이다.

옛 방식: 한 토큰짜리 목표

지금까지의 메모리는 “지금 토큰의 key k_t 를 넣으면 value v_t 가 나오도록” 메모리를 조금 갱신했다. 즉 목표 (loss) 가 토큰 하나짜리였다.

$$\ell(W; k_t, v_t) = \left\| \mathcal{M}(k_t; W) - v_t \right\|_2^2$$

기호 풀이 W = 메모리 (작은 모델) 의 내부 weights, 문맥이 바뀌면 갱신되는 대상. $\mathcal{M}(k; W)$ = key k 를 넣었을 때 메모리가 내놓는 예측값. $k_t = t$ 번째 토큰의 key (질의 열쇠), v_t = 그 토큰이 저장하려는 value (내용물). $\|\cdot\|_2^2$ = 예측과 정답의 차이를 제공해 더한 오차. ℓ = 이 토큰 하나에 대한 오차 (= surprise, 얼마나 틀렸나).

이 식은 이런 뜻. “지금 이 토큰 하나만 놓고, 넣으면 제대로 나오도록 메모리를 고쳐라.” — 딱 하나만 본다.

새 방식: 최근 c 개를 한꺼번에

Omega rule 은 목표를 최근 c 개 토큰의 (가중) 합으로 바꾼다.

$$\min_W \sum_{i=t-c+1}^t \gamma_{t,i} \left\| \mathcal{M}(k_i; W) - v_i \right\|_2^2$$

기호 풀이 c = window 길이 (한꺼번에 볼 최근 토큰 수; $c = 1$ 이면 옛 방식). i = window 안의 토큰 번호 ($t - c + 1$ 부터 지금 t 까지). $\gamma_{t,i} \in [0, 1]$ = **window gate** — step t 에서 i 번째 토큰을 최적화에 얼마나 넣을지 정하는 문 (門). $\gamma_{t,i} \rightarrow 1$ 이면 온전히 포함, $\gamma_{t,i} \rightarrow 0$ 이면 **아예 잘라냄** (in-context pruning). 나머지 기호는 위와 동일.

이 식은 이런 뜻. “지금 토큰 하나가 아니라, 최근 c 개 토큰을 **동시에** 잘 저장하도록 메모리를 고쳐라. 단 그중 필요 없는 토큰은 gate 로 빼도 된다.”

직관 surprise 가 “토큰 하나가 얼마나 틀렸나” 였다면, Omega rule 의 gradient 는 “최근 문맥 덩어리가 얼마나 틀렸나” — surprise of the context 다. Titans 의 momentary/past surprise 에 이어 붙는 세 번째 형태다.

비유 옛 방식이 “받아쓰기 마지막 한 줄만 다시 확인” 이라면, Omega rule 은 “**최근 몇 줄을 묶어서 한꺼번에 교정 (batch write-repair)**” 이다. 그리고 gate $\gamma_{t,i}$ 는 admission control (입장 심사) — “이 토큰은 애초에 받아적을 가치가 있나?” 를 학습으로 정한다. (앞 권의 retention gate α 는 이미 적은 것 중 무엇을 지울지 (eviction) 를 정했다. 둘은 방향이 다른 별개의 문이다.)

window 를 키우면 무슨 일이 생기나

c 를 키우면 의존 범위가 넓어진다. 아래 그림은 sliding-window attention (SWA) 과 Atlas 를 나란히 놓고, c 가 커질수록 ($1 \rightarrow 4 \rightarrow 7$) 각 토큰이 참조하는 과거가 어떻게 변지는지 보여준다.

핵심 대비: SWA 는 window 안을 매번 새로 훑는 비모수 (non-parametric) 방식이고, Omega rule 은 같은 window 를 작은 모델의 weights 에 눌러 담은 parametric 방식이다. 같은 문제를 서로다른 방식으로 푸는 “쌍둥이” 인 셈이다.

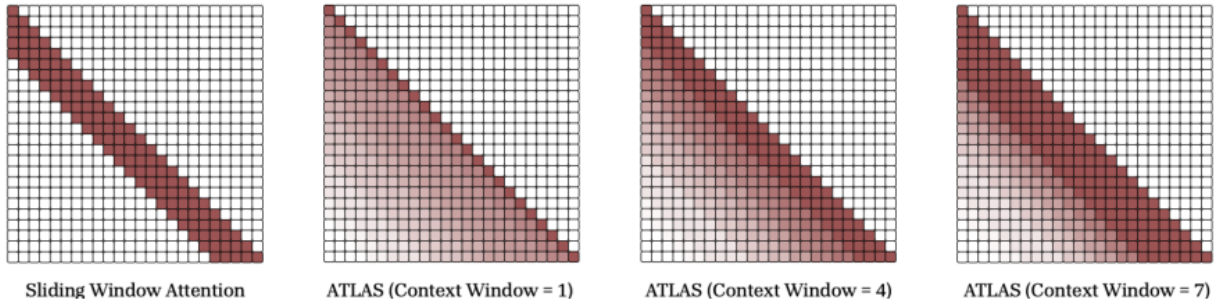


그림 2 — SWA(맨왼쪽) 와 Atlas 의 토큰 의존 구조 비교 (causal mask). SWA 는 폭이 고정된 띠 (banded) 모양으로 인접 c 개만 본다. Atlas 는 window c 를 키울수록 의존이 아래 삼각형 전체로 번지는데, windowed loss 의 gradient 가 이전 상태로 전파되기 때문이다. SWA 는 그 폭안을 비모수로 훑고, Omega rule 은 같은 폭을 parametric memory 에 누적한다. 출처: 저자, Atlas(arXiv:2505.23735) Fig.2 — 원저자 그림 (본서 결과 아님).

계보 회수 — 옛 규칙들이 특수 사례로들어온다

- $c = 1$ 이면 옛날 delta rule(토큰 하나). 여기에 momentum 을 더하면 **Titans** 가 정확히 나온다. 즉 Titans 는 Omega 의 “window=1” 특수사례다.
- $c =$ 문맥 전체, gate 전부 1 이면 문맥 전체에 대한 least-squares가된다 (Mesa-layer). 정확하지만 병렬화가 안 되고 무엇을 버릴지 고를 수없다.

Atlas 는 이 둘 사이의 **중간해**를 고른다: window 라서 step 당 gate 가딱 c 개 (상수) 면 되고, 그래서 학습 가능한 hard gate(γ) 가감당된다.

시스템 모델링 관점 Omega rule 의 state 전이를 열어 보면 정체가 드러난다. linear memory 에서한 step 의 write 는

$$W_t = W_{t-1} \left(\alpha_t I - \sum_{i=t-c+1}^t \gamma_{t,i} \phi(k_i) \phi(k_i)^\top \right) + \sum_{i=t-c+1}^t \gamma_{t,i} v_i \phi(k_i)^\top$$

다. DeltaNet 의 전이가 $W_{t-1}(I - \eta_t k_t k_t^\top) - \mathbf{rank-1}$ 수정이었다면, Omega rule 은 c 개 key 의 합이라 **rank- c** 수정이다. - **state**: 메모리 weights W 하나 (크기는 § 5 에서 회계). 문맥 길이와 무관한 고정 크기. - **토큰당연산**: rank-1 outer product c 개 $\rightarrow$ GEMM 으로 묶으면 내적 차원이 c 인 write GEMM 한 방. 즉 window 를 키우는 것 = **write GEMM 의내적 차원 c 를 키우는 것**. - **데이터 의존성**: recurrence 는여전히 $W_{t-1} \rightarrow W_t$ 한 줄. window 는 이 한 줄의 write 를 “두껍게” 만들 뿐 새 순차 사슬을 만들지 않는다. - **KV cache 대비**: KV 는 append-only(과거 불변) 인데, 이 write 는 read-modify-write(과거를 다시쓰). 캐시 배치 전략이 근본적으로 다르다.

3. 해법 2 — feature map: 메모리 용량을키우기

두 번째 손잡이는 **얼마나 많이 저장할 수 있**나다. Atlas 는 이 계열최초로 “memory capacity” 를 딱 잘라 정의했다: 메모리가 오차 0 으로 정확히저장할 수 있는, 서로 다른 (key, value) 쌍의 최대 개수 m .

나쁜 소식: 그냥 크게 만든다고 용량이 안는다

matrix 메모리 (W 가 $d_v \times d_k$ 행렬) 로 저장하면 용량은 최대 $O(d_k)$ 쌍이다.

기호 풀이 d_k = key 의 차원 (열쇠 벡터 길이), d_v = value 의 차원. W 의 파라미터 수는 $d_k \times d_v$. 용량 m = 정확히 저장 가능한 쌍의 수.

이 식은 이런 뜻. 파라미터를 $d_k \times d_v$ 개나 썼는데 저장칸은 d_k 개뿐이다. 즉 **파라미터를 늘리는 것 $\neq$ 용량을 늘리는 것**. (예: d_v 를 d_k 만큼 키우면 파라미터는 제곱으로 느는데 용량은 여전히 $\sqrt{\text{파라미터}}$ 급.)

메모리를 깊은 MLP 로 만들면 (depth $\uparrow$) 용량이 좀 오르지만, 여전히 (d_k, d_v) 에 대해 subquadratic 이라 근본 한계는 못 넘는다.

좋은 소식: key 를 더 큰 공간으로 “올린다”

용량 상한은 결국 key 가 사는 공간의 차원이 정한다. 그래서 key 를 그대로 쓰지 않고 **feature map** ϕ 로 더 큰 공간으로 밀어 올린다.

polynomial feature map $\phi_p(x)$ = 차수 p 이하의 모든 monomial(곱항) 을 쌓은 벡터. 이러면 올라간 차원이 $D = \binom{d_k+p}{p} = \Theta(d_k^p)$ 가 되고, 용량도 $O(d_k^p)$ 로 뿔다.

기호 풀이 ϕ = feature map(key 를 더 큰 공간으로 올리는 함수). p = 차수 (올리는 강도; p 가 클수록 더 큰 공간). D = 올라간 뒤의 차원 ($\Theta(d_k^p)$). $\phi^* = p \rightarrow \infty$ 의 극한 map.

비유 원래 종이가 1 차원 줄 (선) 이라 점을 몇 개밖에 못 찍었다. ϕ 는 그 줄을 넓은 평면 · 입체로 펼치는 것이다. 공간이 넓어지니 겹치지 않게 찍을 수 있는 점 (= 저장할 연상) 이 훨씬 많아진다. 차수 p 가 “몇차원까지 펼치나” 를 정한다.

직관 이 capacity 상한은 **optimizer 와 무관**하다. 어떤 방법으로 갱신하든 D 쌍을 못 넘는다. 그래서 feature map 은 “용량 천장 자체를 옮기는 손잡이” 이고, 뒤에 나올 Muon 은 “그 천장 안에서 실제 도달 품질을 올리는 손잡이” 다 — 두 축이 직교한다.

극한: softmax attention 은 용량이 무한한 메모리였다

p 를 무한대로 보내면 ϕ^* 가 되고, 이때 $\exp(q^\top k) = \phi^*(q)^\top \phi^*(k)$ 가 **정확히** 성립한다. 즉 softmax attention 은 무한 차원 공간 위의 associative memory 이고, **용량이 unbounded(무한)** 다.

핵심 “attention 이 긴 문맥 recall 에서 고정 state 모델을 이기는 이유” 가여기서 정리로 확정된다 — 신비가 아니라 **용량 천장의 차이**다. KV cache = 압축하지 않는 (= 용량 무한) 메모리, 고정 state = 압축하는 (= 용량 유한) 메모리. Atlas 의 feature map 은 유한 메모리의 천장을 $O(d_k^p)$ 까지 끌어올려 그 격차를 좁히려는 시도다.

한계 논문은 실제로 쓴 차수 p 와 sketch(차원 줄이는 근사) 방법을 공개하지 않는다. 그래서 “용량이 얼마나 늘었나”는 방향은 알아도 절대값은 계산할 수 없다. 이 미공개가 § 5의 state 회계를 반쪽짜리로 만든다.

4. 해법 3 — Muon: 더 똑똑한 갱신

세 번째 손잡이는 **어떻게 갱신하나**다. Omega rule과 feature map을 다 갖춰도, 갱신을 plain gradient descent(1차)로 하면 나쁜 지점에 눌러앉을 수 있다. Atlas는 같은 목표를 **Muon**이라는 optimizer로 갱신한다.

$$S_t = \beta_t S_{t-1} - \nabla_W(\text{windowed loss}), \quad W_t = \alpha_t W_{t-1} + \eta_t \text{NS}_\kappa(S_t)$$

기호 풀이 S_t = momentum buffer(최근 놀란 방향들을 쌓아 둔, W 와 같은 크기의 텐서). β_t = momentum decay gate(옛 관성을 얼마나 남길지). $\nabla_W(\dots)$ = windowed loss의 gradient(이번에 어느 방향으로 고쳐야 하나). α_t = retention gate(옛 메모리를 얼마나 남길지, forgetting). η_t = learning-rate gate(이번 걸음의 보폭). NS_κ = Newton-Schulz 반복 κ 회 — 행렬을 “정규직교에 가깝게” 다듬는 연산.

이 식은 이런 뜻. (1) 최근 놀란 방향들을 관성으로 쌓고 (S_t), (2) 그 관성을 NS_κ 로 한 방향으로 쏠리지 않게 골고루 편 다음, (3) 옛 메모리를 조금 잊으면서 (α_t) 그 방향으로 한 걸음 간다.

직관 momentum = “최근 놀란 방향의 관성”(→ Titans에서 온 것). Muon이 더한 것은 NS_κ 다: raw momentum을 그대로 쓰면 특정 방향으로만 크게 쏠리는데, NS_κ 는 그 쏠림(singular value)을 균등하게 퍼서 **모든 방향으로 고르게** 갱신한다. 논문은 이걸 “2차 정보의 근사”로 읽는다 — 1차는 방향만, 2차는 “어느 방향이 더 급한지”까지 본다.

비유 gradient descent가 “가장 가파른 쪽으로만 성큼 내려가는” 하산이라면, Muon은 “발을 좌우로 고르게 디디며(orthogonalize) 미끄러지지 않게 내려가는” 하산이다. 좁은 계곡에서 한쪽으로 튕겨나가는 걸 막는다.

κ — 새로운 종류의 손잡이 (test-time compute dial)

NS_κ 의 반복 횟수 κ 는 특이하다. 논문은 이걸 “**internal test-time compute parameter**”라 부른다: κ 를 늘리면 갱신이 더 정교해지고(주장상) 기억 품질이 좋아지되, decode 연산량(FLOPs)이 선형으로 는다. 실전값은 $\kappa = 5$ (“NS-5”).

시스템 모델링 관점 κ 는 state 크기를 전혀 바꾸지 않고 decode 연산량 ↔ 품질을 교환하는 layer 내부 dial이다. speculative decoding이 요청 단위의 품질/지연 트레이드라면, κ 는 layer 내부의 트레이드다. 토큰당 추가 연산은 weight 행렬당 κ 회 반복 × 반

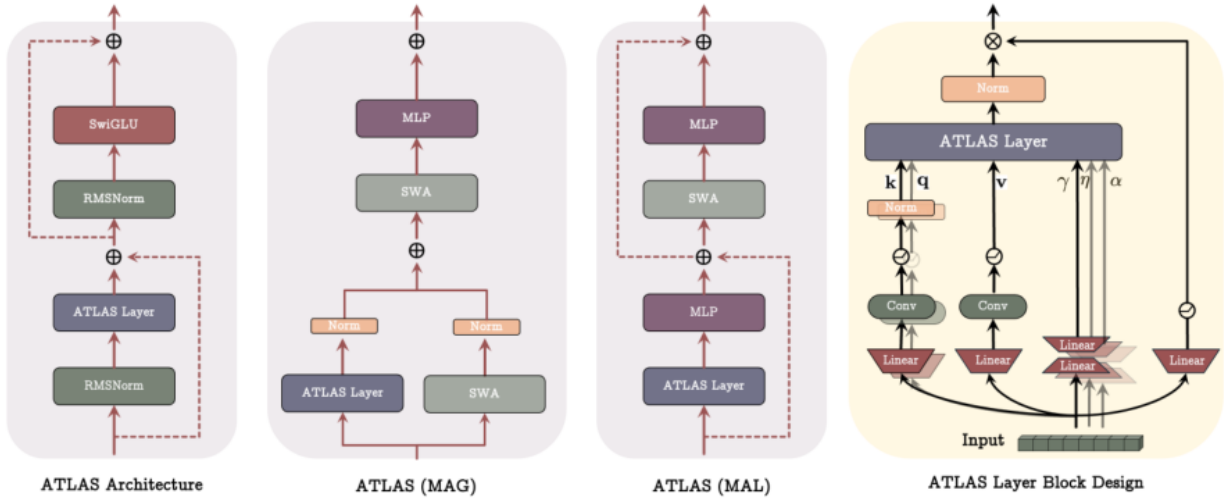


그림 3 — Atlas 아키텍처 (맨왼쪽) 와 두 hybrid 구성 (MAG·MAL), Atlas layer 의 block 설계 (맨오른쪽). 입력에서 Q/K/V projection(linear + 크기-4 short conv) 과 세 gate(gamma, eta, alpha) 의 producer 가 갈라져 나와 ATLAS Layer 로 들어간다. (도해에는 feature map 차수 $p \cdot sketch$ 차원 · memory head 분할이 표기되지 않는다.) 출처: 저자, Atlas(arXiv:2505.23735) Fig.3 — 원저자 그림 (본서 결과 아님).

복마다 서너 개의 정방 matmul $\approx$ 15-20 개의 추가 matmul($\kappa = 5$ 기준). chunk 안 전위치에 batch 되므로 GEMM shape 는 좋다. 단 주의: 품질이 κ 에 대해 실제로 단조 증가하는지는 논문에 **측정이 없다** — 지금은 “존재하는 knob” 이지 “검증된 knob” 이 아니다.

주의 ablation 을 정직하게 읽으면, 760M 스케일에서 **Muon** 을 빼면 **perplexity** 가 오히려 좋아진다 ($19.65 < 19.97$). Muon 이 산 것은 reasoning accuracy 0.21pt 뿐이다. “optimally memorize” 의 optimality 는 대부분 window 와 capacity 축이 벌어진 것이고, Muon(=locally optimal) 의 값은 과제 의존적이며 아직 미결이다.

5. Atlas 아키텍처 한 장으로 + 시스템회계

세 손잡이를 엮은 layer 의 배선과, 두 hybrid 구성 (SWA 와 결합하는 MAG, memory 뒤에 SWA 를 붙이는 MAL) 을 논문이 한 장에 그려 둔다.

기본 메모리는 2 층 residual MLP $\mathcal{M}(z; W) = z + W_1 \sigma(W_2 z)$ 이고, Atlas++는 이를 gated MLP 로 올린 변형이다. 합성 문법 (MAG/MAL/MAC) 은 Titans 에서 그대로 상속한다.

여기서 반드시 기억할 것: 무엇이 언제 학습되나. “메모리가 test time 에 훈련된다” 는 말은 **pre-trained checkpoint** 가 변한다는 뜻이 아니다. 두 loop 로 나뉜다.

- **outer loop(pre-training):** projection(W_Q, W_K, W_V), gate producer($\alpha, \eta, \beta, \gamma$ 를 만드는 함수), feature 계수, 메모리 초기 상태 W_{init} 를 **한 번** 학습한다. 이후 frozen.

- **inner loop(inference)**: 메모리 weights W_t 와 momentum S_t 만, 매 문맥마다 W_{init} 에서 다시 시작해 갱신한다. **문맥이 끝나면 폐기**.

핵심 gate 값은 매 토큰 바뀌지만, gate 를 만드는 함수는 frozen 이다. inference 에서 바뀌는 건 메모리 내용물 (W_t, S_t) 뿐. 그래서 Atlas 는 이걸 “test-time training” 이 아니라 “**test-time memorization**”(저장 · 인출일 뿐 학습이 아님) 이라 부른다.

state 크기의 청구서 — 용량 ↑ 은 무엇으로 지불하나

용량을 키운 대가는 반드시 어딘가로 청구된다. per-layer state 는 세팅어리다.

state 덩어리	크기 (원소 수)	문맥 의존?
메모리 weights W (표준형 W_1, W_2)	$\approx 8d_m^2$	무관
momentum S (같은 shape)	$\approx 8d_m^2$	무관
window buffer(최근 c 개 $\phi(k), v$)	$O(c(d_k + d_v))$	무관 (무시 가능)

기호 풀이 d_m = memory 폭 (메모리 MLP 의 hidden 크기). 표준형은 expansion 4 의 2 층이라 W_1, W_2 합쳐 $\approx 8d_m^2$ 원소. momentum 도 같은 shape 라 또 $8d_m^2$. 합계 $\approx 16d_m^2$ 이며 **문맥 길이와 무관**. 대조로 Transformer KV cache 는 layer 당 $2L_{\text{ctx}}d$ — 문맥 길이 L_{ctx} 에 비례.

시스템 모델링 관점 두 state 가 같아지는 **crossover** 문맥 길이를 잡으면 $L^* = 16d_m^2/(2d)$ 다. memory 폭을 model 폭과 같게 잡으면 ($d_m = d$) $L^* = 8d = 1.3\text{B}$ 구성 ($d = 2048$) 에서 약 **16K 토큰**. 즉: - 문맥 < 16K: KV cache 가 더 작다 (Atlas state 가 손해). - 문맥 > 16K, 특히 BABILong 의 10M 구간: Atlas state 가 KV cache 의 수백분의 일.

이것이 “용량은 cache 성장이 아니라 **state byte 와 matmul 폭으로 지불된다**” 의 정확한 뜻이다. 단 청구서 액수엔 미지수가 둘: (1) 실제 d_m 과 head 분할이 논문에 없고, (2) ϕ_p 가 메모리 첫 층 입력폭을 $\Theta(d_k^p)$ 로 불리는데 **구현 차수 p · sketch 차원이미공개** 라 절대값은 계산 불가.

우리 스터디의 자체 실험도 같은 crossover 구조를 보여준다 — state 크기가 클수록 (memory 폭 $d_m \uparrow$) TTT 가 KV 를 이기는 지점이 더 긴 문맥으로 밀린다.

decode 의 state 트랙픽

시스템 모델링 관점 매 토큰, 각 layer 의 메모리는 **read-modify-write** 다: W 와 S 를 읽고 $\rightarrow$ gradient · momentum · NS 계산 $\rightarrow$ 둘 다 다시 쓴다. 토큰당 트랙픽 $\approx 2 \times 16d_m^2$ 원소의 읽기+쓰기. DeltaNet 류의 단순 $d \times d$ matrix state 대비 원소 수로 **16 배 급** (expansion 4 의 2 층 + momentum 때문). 그리고 ϕ_p lift 가 있으면 첫 층 입력이 $D = \binom{d_k+p}{p}$ 로 더 커진다. KV cache 와 달리 **append-only 가 아니라 RMW** 라서, 캐시 계층 상주 전략이 완전히 다르다. 연산 성격은 전부 dense matmul(banded gradient

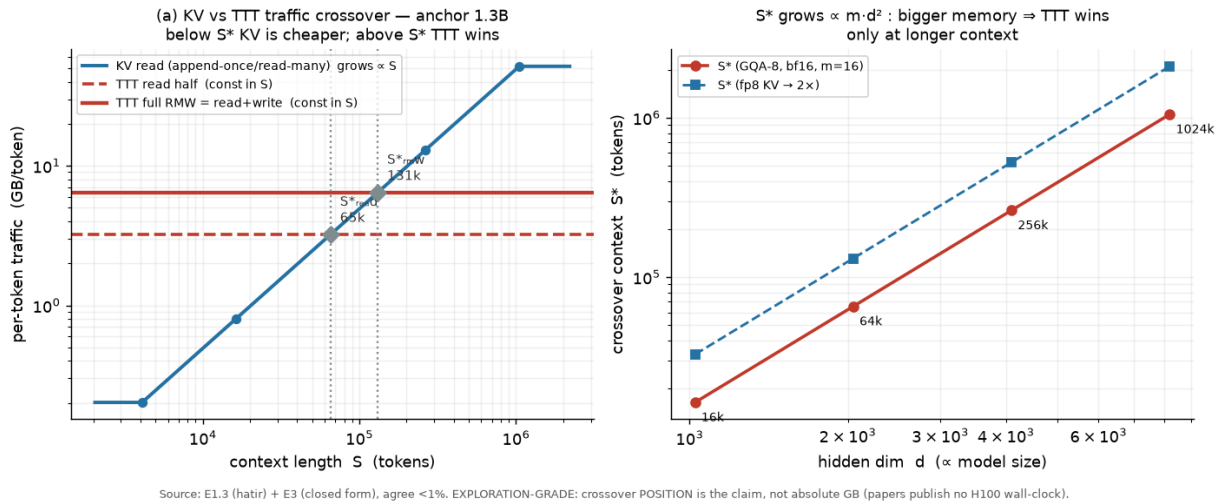


그림 4 — (왼쪽) 문맥 길이 S 에 대한 토큰당 트래픽. KV read는 S 에 비례해 늘고 (파랑), TTT의 read-modify-write는 S 와 무관한 상수 (빨강) 라 특정 S 에서 교차한다. (오른쪽) hidden dim d 가 클수록 crossover S 가 길어진다 ($S^* \propto m \cdot d^2$). 출처: 본 스터디 자체 실험 (exploration-grade) — crossover 위치가 요지이며 절대 GB 값이 아님.

einsum + momentum broadcast scan + batched NS-5) 이라 GEMM 친화적이다.

주의 (batching). 요청마다 다른 fast weights(W_t, S_t)를 쓰므로 shared-weight batching이 깨진다 — decode가 grouped-GEMM 형태가 된다. MAG/MAL hybrid는 SWA 브랜치를 달고 있어 rolling KV cache와 recurrent state를 **동시에** 관리해야 한다.

6. 실험이 말하는 것 (요점만)

- **긴 문맥이 헤드라인.** BABILong에서 Atlas는 다른 recurrent baseline이 무너지는 10M 토큰 구간에서 **+80% accuracy**를 유지한다 — 4K 훈련 문맥의 수천 배 외삽. window(objective) + polynomial kernel(capacity) + Muon(관리)의 합작으로 귀속.
- **needle-in-haystack(S-NIAH).** 4K로 훈련한 모델이 16K에서도 강하다 (Atlas 84.0 vs Titans 80.2; Dot·MAG는 93–100).
- **language modeling.** 1.3B/100B tokens에서 Atlas가 Titans·Gated DeltaNet·Transformer++를 앞선다.
- **retrieval은 아직 진다 (의무 caveat).** in-context recall 평균에서 **Transformer**가 여전히 이긴다 (53.55 vs Atlas 43.70). Atlas는 recurrent 중 최고지만 gap을 좁혔을 뿐 단지 못했다 — §3의 capacity 이론이 예측하는 방향 그대로다 (무한 용량 attention vs 유한 state).

주의 (스케일 상한). 모든 품질 주장은 1.3B/100B tokens에서 끝난다. 그리고 훈련이든 decode든 **wall-clock** 수치가 한 건도 없다 — “overhead 없다”, “병렬화된다”는 전부 구조 논증이지 측정이 아니다.

7. 한계

- **retrieval gap 미해소**: 남은 격차가 용량 한계인지, inner 최적화한계인지, 유한 state 의 본질인지 이 논문으로는 판별 안 됨.
- **Muon 의 값이 애매**: perplexity 기준으론 오히려 손해. κ -품질 곡선 미측정.
- **feature map 구현 미공개**: 실제 p -sketch 결과 차원이 없어 state-FLOPs 회계가 절 대값으로 불가.
- **unnormalized 일반화**: Transformer 를 다시 유도한 DeepTransformers/Dot 은 softmax 분모를 버린 채 분석됨.
- **chunk-size mismatch 방치**: 훈련은 큰 chunk C , decode 는 $C = 1$ 인데 그 괴리를 논점 삼지 않음 — 다음 권의 출발점.

요약 Atlas = Titans/Miras 에 세 손잡이를 얹은 모델. **Omega rule**(최근 c 개를 한꺼 번에 기억 — rank- c write), **feature map**(key 를 큰 공간으로 올려 용량 천장을 $O(d_k^p)$ 로 $\uparrow$, 극한은 softmax=무한 용량), **Muon**(momentum 을 NS_κ 로 골고루 퍼는 근사 2 차 갱신). 시스템 관점의 결론: 용량을 키우는 대가는 KV cache 성장이 아니라 **고정 크기 state 의 byte 수** ($\approx 16d_m^2$) **와 matmul 폭으로 지불되며**, decode 는 append 가 아닌 read-modify-write 다. 긴 문맥에서 크게 이기지만 순수 retrieval 은 아직 Transformer 에 진다.

다음 권 예고. G06 · TNT — Atlas 가 방치한 “훈련 chunk C 와 decode chunk 의 mismatch” 를 정면으로 발견하고, 계층적 memory 와 주기적 reset 으로 “그걸 만들고 돌리는 데 얼마가 드는가” 를 다시 설계한다.